The Computational Power of Catalytic Comparator Circuits

Project Goal and Importance

This project investigates what happens when two opposing computational constraints are combined: taking away a programs ability to copy information while simultaneously giving it access to more memory.

Modern computing increasingly demands memory efficiency under tight resource constraints. Recent work (Buhrman et al. 2014) has shown a counter-intuitive method to do this: a program can use memory that is already full, say a hard drive containing someone else's data, as working space during computation, provided it restores that memory to its original state before finishing. This is catalytic computation, and remarkably, even this borrowed, must-be-returned memory can expand what a program can compute.

Comparator circuits (Subramanian 1990) represent the opposite kind of constraint: they are circuits that can only compare and swap pairs of bits. An important feature of this model is that it does not allow copying data: once two bits are compared, the original values are gone. This means comparator circuits are fundamentally limited in how they can reuse information. The class of problems solvable by such circuits is called CC.

This project lies at the intersection of these two ideas, and aims to analyse a comparator circuit analogue of catalytic computing. When a comparator circuit is equipped with catalytic auxiliary memory, does that memory actually help? Can it compensate for the inability to copy bits, or does the obligation to restore the memory ultimately neutralize any advantage it might offer? The goal of this project is to characterize the computational power of catalytic comparator circuits (the complexity class CC_{cat}) and determine whether it is strictly more powerful than CC, or whether the two classes coincide.

This work is motivated in part by the growing importance of computation under resource constraints. Edge computing devices, such as IoT sensors, security cameras, and embedded systems, operate under tight memory limitations, often with pre-loaded firmware or data occupying much of their available storage. Catalytic computation models precisely this setting: a device that must compute using memory that is already full, without permanently altering its contents. As security-critical tasks increasingly migrate to such devices, understanding the theoretical limits of catalytic models becomes practically relevant—it informs what kinds of computations are feasible in memory-constrained environments and where fundamental barriers lie.

Comparator circuits capture the notion of "computation without copying," and understanding them helps isolate the role that data duplication plays in computational power. Catalytic computation introduces a complementary constraint: it augments a machine's available memory, but only under the obligation to preserve it. Studying CC_{cat} allows us to examine both constraints simultaneously, asking what happens when we remove the ability to copy *and* add the ability to borrow. Understanding CC_{cat} also has broader significance within theoretical computer science. The class CC sits in a poorly understood region of the complexity landscape, and its internal structure (what problems it contains, and why) remains largely mysterious. Studying how a catalytic resource interacts with the comparator model may shed new light on this structure, and the techniques developed here may transfer to other circuit models where similar questions are open.

Relevant Literature

The space-complexity of a program measures how much memory it requires during computation. Comparator circuits were introduced by Subramanian (1990) as a restricted model of computation in which the only allowed operation is a compare-and-swap on

pairs of bits. Despite this severe restriction, the class of problems solvable by such circuits (CC) turns out to be surprisingly rich. Mayr and Subramanian (1992) showed that CC contains all problems solvable in nondeterministic logspace (NL), and is itself contained in polynomial time (P), placing it in a poorly understood middle ground. Among the natural problems that are CC-complete (meaning they are among the hardest problems in the class) is the stable marriage problem, the task of pairing n men and n women according to their preferences so that no two people would rather be with each other than their assigned partners. Cook, Filmus, and Lê (2014) later strengthened our understanding of CC by giving several equivalent characterizations of the class and providing evidence that CC is genuinely distinct from other known classes in the same range. A key technical result due to Razborov (1992) shows that a natural family of complexity measures suited to comparator circuits can never exceed $O(n)$ on n-bit functions, a fundamental limitation on available lower bound techniques for this model.

Catalytic computation was introduced by Buhrman et al. (2014) and is motivated by a simple question: can a program make productive use of memory that is already full, as long as it promises to leave that memory exactly as it found it? Surprisingly, the answer is yes, and the additional power this grants is substantial. Even under this strict restore condition, access to a full auxiliary memory tape allows logspace machines to solve problems previously thought to require far greater resources.

The practical relevance of this model has grown with recent results: Cook and Mertz (2024) used catalytic techniques to solve the Tree Evaluation Problem (a benchmark problem central to the question of whether every problem solvable in polynomial time can also be solved in logarithmic space) using dramatically less space than previously known. Building on this, Williams (2025) showed that any program running in time t can be simulated using only $O(\sqrt{t \log t})$ space.

The direct predecessor of this project is the work of Robere and Zuiddam (2021), who asked what happens when comparator circuits are given catalytic auxiliary memory. They define the catalytic comparator circuit complexity $CC_{\text{cat}}(f)$ as the size of the smallest comparator circuit that computes f with the help of a catalyst, and show that it is sandwiched between two quantities that are already understood:

$$\widetilde{CC}(f) \leq CC_{\text{cat}}(f) \leq CC(f),$$

where $\widetilde{CC}(f)$ is the amortized comparator complexity of f, that is, roughly speaking, the average cost of computing many copies of f simultaneously. They further show that $CC_{\text{cat}}(f)$ can be characterized as the solution to an optimization problem, and that the lower end of the sandwich, $\widetilde{CC}(f)$, is always at most $O(n)$. However, these results leave the most important question unanswered: does the catalyst actually make comparator circuits more powerful, or does CC_{cat} coincide with CC? This project takes that question as its starting point.

Project Methodology

This project is theoretical in nature. The first phase consists of a targeted literature review, focused on two specific technical threads directly relevant to the project. The first is the connection between comparator circuits and toggle machines—physical switching devices such as the Digi-Comp II, in which balls roll through a network of toggles that alternate direction on each pass (Aaronson 2014). This correspondence is useful not merely as an analogy. A key structural property of comparator circuits is that they are 1-Lipschitz with respect to Hamming distance, that is changing a single input bit can affect at most one output bit. Tracing a change through the circuit inductively, there is no amplification, no

sensitive dependence on initial conditions. The Digi-Comp II makes this immediately visible: a ball is a physical object that can only travel one path, so a single change to the input can only affect one ball's trajectory. In a circuit diagram, although the same property holds, it is not obvious and requires an inductive argument tracing the change gate by gate. Having a physical, intuitive model of this stability property provides an alternative perspective on the structure of CC, and alternative perspectives are often the key to progress on hard problems.

The second thread is the mathematical machinery underlying modern results in catalytic computation, particularly the proof that $BPL \subseteq CL$ (Buhrman et al. 2014), the Cook–Mertz tree evaluation result Cook and Mertz (2024), and the amortized circuit complexity framework of Robere and Zuiddam (2021). Understanding the techniques in these papers is essential, since any progress on CC_{cat} will likely draw on or adapt them.

The subsequent research phase does not follow a fixed plan, which is in the nature of theoretical research. The process will consist of formulating candidate approaches, testing them on known CC-complete problems such as stable matching, and either pursuing or abandoning them based on what emerges. A key question will be how a catalytic comparator circuit can track enough information to restore the auxiliary memory to its original state upon completion, given the no-copying constraint of the model. To complement the analytical work, computational simulator will be implemented to enumerate catalytic comparator circuits and instantiate the Robere–Zuiddam linear program using a standard solver. The goal is to identify patterns in the computational output that suggest or rule out a proof that $CC_{\text{cat}} = CC$. Ideas that survive initial scrutiny will be written up rigorously as they develop, both to consolidate understanding and to produce material that can be refined into a final report.

References

Aaronson, Scott. 2014. *The Power of the Digi-Comp II.* Shtetl-Optimized (blog), https://scottaaronson.blog/?p=1902. Accessed 2025.

Buhrman, Harry, Richard Cleve, Michal Koucký, Bruno Loff, and Florian Speelman. 2014. "Computing with a Full Memory: Catalytic Space." In *Proceedings of the 46th Annual ACM Symposium on Theory of Computing (STOC)*, 857–866.

Cook, James, and Ian Mertz. 2024. "Tree Evaluation Is in Space $O(\log n \cdot \log \log n)$." In *Proceedings of the 56th Annual ACM Symposium on Theory of Computing (STOC)*, 1268–1278.

Cook, Stephen A., Yuval Filmus, and Dai Tri Man Lê. 2014. "The Complexity of the Comparator Circuit Value Problem." *ACM Transactions on Computation Theory* 6 (4): 1–44.

Mayr, Ernst W., and Ashok Subramanian. 1992. "The Complexity of Circuit Value and Network Stability." *Journal of Computer and System Sciences* 44 (2): 302–323.

Razborov, Alexander A. 1992. "On Submodular Complexity Measures." In *Boolean Function Complexity*, edited by Mike S. Paterson, 169:76–83. London Mathematical Society Lecture Note Series. Cambridge University Press.

Robere, Robert, and Jeroen Zuiddam. 2021. "Amortized Circuit Complexity, Formal Complexity Measures, and Catalytic Algorithms." In *Proceedings of the 62nd IEEE Annual Symposium on Foundations of Computer Science (FOCS)*, 759–769.

Subramanian, Ashok. 1990. "The Computational Complexity of the Circuit Value and Network Stability Problems." Technical Report STAN-CS-90-1339. PhD diss., Stanford University.

Williams, Ryan. 2025. "Simulating Time With Square-Root Space." In *Proceedings of the 57th Annual ACM Symposium on Theory of Computing (STOC).*

Timeline

Week 1 (June 21 – June 27): Background Reading on Comparator Circuits and Toggle Machines.

Review foundational literature on comparator circuits and the complexity class CC. Study the connection between comparator circuits and the Digi-Comp II toggle device, focusing on the 1-Lipschitz property and what it reveals about the structural constraints of the model.

Week 2 (June 28 – July 4): Catalytic Computation and Simulator Development.

Read Buhrman et al. (2014) in depth, focusing on the techniques underlying the proof that $BPL \subseteq CL$. Begin implementing a computational simulator for catalytic comparator circuits, representing states as tuples of bits and applying compare-and-swap operations, as a way of building concrete intuition for the model alongside the reading.

Week 3 (July 5 – July 11): Study of the Amortized Circuit Complexity Framework.

Read Cook and Mertz (2024) and Robere and Zuiddam (2021) carefully, with particular attention to the linear programming duality framework and the characterization of CC_{cat} as the integer optimum of a natural LP. Complete the simulator and extend it to instantiate the Robere–Zuiddam linear program using a linear programming solver.

Week 4 (July 12 – July 18): Consolidation and Research Planning.

Consolidate notes from the reading phase and verify the simulator against known cases. Identify the specific techniques most likely to be relevant, write up a summary of available tools, and settle on an initial research direction to pursue.

Week 5 (July 19 – July 25): Investigation of CC-Complete Problems in the Catalytic

Setting.

Examine known CC-complete problems, including stable matching and lexicographically-first maximal matching, and investigate what catalytic comparator circuits solving them look like. Focus on the information-tracking question: how does the circuit preserve enough state to restore the catalyst upon completion. Use the simulator to run experiments and generate data to guide the analytical work.

Week 6 (July 26 – August 1): Analysis of the Restoration Mechanism and LP Instantiation.

Focus on the information-tracking question: how does the circuit preserve enough state to restore the catalyst upon completion. Use the simulator to instantiate the Robere–Zuiddam linear program for concrete cases and solve it using a standard solver, examining whether the integer optimum coincides with the fractional optimum.

Week 7 (August 2 – August 8): Pattern Identification and Proof Strategy.

Analyze the computational output from the simulator to identify structural patterns across different functions and catalyst sizes. Use these patterns to formulate or refine analytical proof strategies, focusing on conditions that would be sufficient to conclude $CC_{\text{cat}} = CC$.

Week 8 (August 9 – August 15): Evaluate Current Directions and Write-Up.

Pursue the most promising direction identified in the preceding weeks. Write up partial results, conjectures, and proof attempts in rigorous mathematical form as they develop.

Week 9 (August 16 – August 22): Continued Research and Result Consolidation.

Continue developing the main research direction. If a proof strategy has emerged, work to complete it. If specific approaches have failed, formalize the obstruction, as negative results also contribute meaningfully to understanding the model.

Week 10 (August 23 – August 29): Final Report.

Compile all results and write a final report documenting the research process, summarizing findings, and identifying directions for future work. Polish exposition and verify all arguments.